SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 40%

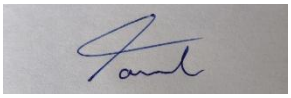
Duration :60 Mins
On Class
QUESTIONS: 5 **Total Marks:50**

Annexure A

CLASS TEST

Code of Conduct:

*I **Tamil Elakiya S (192524443)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Q. No	Understanding of Concepts (25)	Explanation (30)	Application (20)	Organization(15)	Accuracy(10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 30**	**/ 20**	**/ 15**	**/ 10**	**/ 100**

Annexure A

CLASS TEST

Q1. Write down the translation scheme to generate code for assignment statement. List the scheme for generating three address code for the assignment statement x = (a + b) * (c + d) (10 Marks)

Q2.Consider the following Boolean expression:

(a > b) && (c < d) || (e < f)

The grammar for Boolean expressions is given as:

B → B1 || B2
B → B1 && B2
B → ! B1
B → E1 relop E2
B → true
B → false

(i). Explain how operator precedence and associativity affect the evaluation of the given Boolean expression.

(ii). Construct the Three Address Code (TAC) for the given Boolean expression using short-circuit evaluation.

(iii). Represent the control flow of the Boolean expression using appropriate labels and conditional jump statements. (10 Marks)

Q3.Given the declaration sequence

int a, b[10];

float c, d;

derive the intermediate representation including symbol table entries, type sizes, and offset calculations. (10 Marks)

Q4.Consider the following program fragments:

(i). while (A < C && B > D) do

```
if (A == 1) then
    C = C + 1
else
    while (A <= D) do
        A = A + B
```

Generate the Three Address Code (TAC) for the above program fragment. Clearly show:

(a) Conditional and unconditional jump statements

(b) Use of labels for control flow

(c) Evaluation of Boolean expressions (05 Marks)

(ii). main()

```
{
    int i;
    int a[10];
    i = 1;
    while (i <= 10)
    {
        a[i] = 0;
        i = i + 1;
    }
}
```

Translate the executable statements of the above C program into Three Address Code (TAC). Clearly indicate:

(a) Representation of array elements

(b) Loop control using labels

(c) Increment and assignment operations (05 Marks)

Q5. Use backpatching to generate intermediate code for:

```
if (a < b && c < d)
 x = a + b;
```

```
else
x = c + d;
```

Show the intermediate instructions before and after backpatching. (10 Marks)

1. Given:

$$x = (a+b) * (c+d)$$

TAC instruction generally has the form:

$$x = y \text{ op } z$$

where:

- x is the result
- y and z are operands
- op is an arithmetic or logical operator.

For an assignment statement:

$$\boxed{x = E}$$

the translation scheme is:

$$\boxed{S \rightarrow id = E}$$

S.code = E.code || emit (id.lexeme = E.place)

Translation scheme:

↳ $E \rightarrow E_1 + E_2$

Semantic action:

E.place = newtemp()
emit (E.place = E1.place + E2.place)

↳ $E \rightarrow E_1 * E_2$

semantic action:

E.place = newtemp()
emit (E.place = E1.place * E2.place)

Given Expression:

$$\boxed{X = (a+b) * (c+d)}$$

Three Address code:

$T_1 = a+b$

$T_2 = c+d$

$T_3 = T_1 + T_2$

$x = T_3$

Explanation:

1). T_1 stores the result of a+b.

2). T_2 stores the result of c+d.

3) T_3 stores the product of T_1 and T_2.

4) The final result is assigned to x.

2.) Given:

(a>b) && (c<d) || (e<f)

i) Precedence order:

Relational operators

↓

&&

↓

||

Both && and || are left associative.

Therefore:

A&&B||C is evaluated as (A&&B)||C

ii) TAC:

```
L1: if a>b goto L2
    goto L4
L2: if c<d goto L3
    goto L4
L3: goto L5
L4: if e<f goto L5
    goto L6
L5: result = TRUE
    goto L7
L6: result = FALSE
L7: END
```

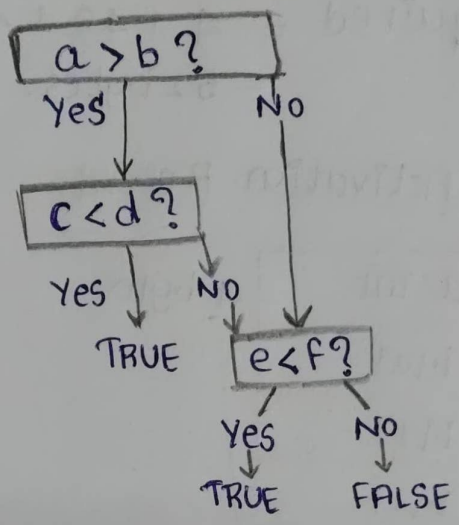
For:

((a>b) && (c<d)) || (e<f)

the execution is:

1. check a>b.
2. If false, directly evaluate e<f.
3. If true, evaluate c<d.
4. If c<d is true, entire expression is true.
5. If c<d is false, evaluate e<f.

iii) Control Flow:



3. Given:

```
int a, b[10];
float c, d;
```

Assume the standard sizes:

int → 4 bytes

float → 4 bytes

Starting offset = 0

Step 1: Calculate the width of each variable

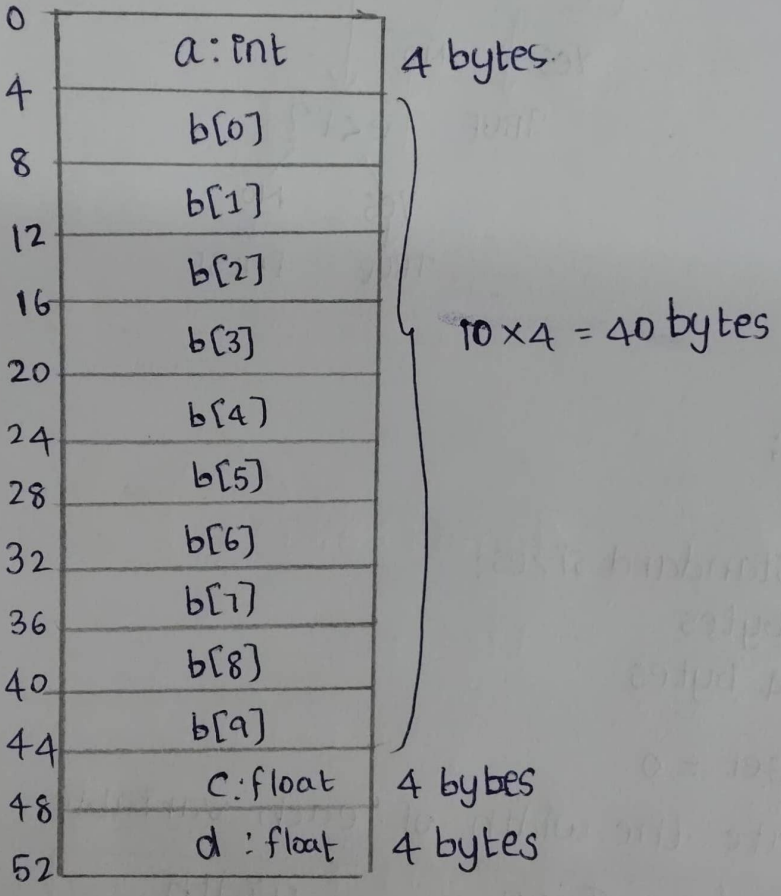
variable	Type	width
a	Integer	4
b	array [10] of int	4 x 10 = 40
c	float	4
d	float	4

Step2: Calculate offset real

variable	Type	width	offset
a	Integer	4	00
b	array[10]int	40	04
c	float	4	44
d	float	4	48

Total storage required = 4 + 40 + 4 + 4
= 52 bytes.

Memory Layout of Activation Record:



Array Address Calculation:

Address (b[i]) = Base (b) + i × width (int)

Therefore:

Address (b[i]) = Base (b) + i × 4

4. i) Given:

```
while (A<C and B>D) do
if A=1 then C = C+1
else while A <=D do
A = A+B
```

TAC:

```
100: if A<C then goto 102
101: goto 115
102: if B>D then goto 104
103: goto 115
104: if A=1 then goto 106
105: goto 109
106: t1 = C+1
107: C := t1
108: goto 100
109: if A <=D then goto 111
110: goto 100
111: t2 = A+B
112: A = t2
113: goto 109
114: goto 100
115: *
```

ii) Given:

```
main(){
    int i;
    int a[10];
    i=1;
    while (i<= 10)
    { a[i] = 0;
      i = i+1;
    }
}
```

TAC:

100: i := 1

101: if i<=10 then goto 103

102: goto 108

103: t_1 := i

104: $t_2 = 4 * t_1$

105: $a[t_2] = 0$

106: i = i+1

107: goto 101

108: Stop.

---

5. Given:

```
if (a<b && c<d)
    x = a+b;
else
    x = c+d;
```

Before Backpatching:

1. if a<b goto —
2. goto —
3. if c<d goto —
4. goto —
5. T_1 = a+b
6. x = T_1
7. goto —
8. T_2 = c+d
9. x = T_2
10. END

Backpatch Lists:

For: (a < b)

we have:

truelist = {1}

falselist = {2}

For: (c < d)

we have:

truelist = {3}

falselist = {4}

So:

backpatch ({1}, 3)

Overall list becomes:-

truelist = {3}

falselist = {2, 4}

After Backpatching:

L1:

1. If a < b goto L2
2. goto L4

L2:

3. If c < d goto L3
4. goto L4

L3:

5. $T_1 = a + b$
6. $x = T_1$
7. goto L5

L4:

8. $T_2 = c + d$
9. $x = T_2$

L5:

10. END